

LINK de acesso do código-fonte:

https://github.com/carlosadauto7/metodos-matematicos-2026.1/blob/main/tarefa_04/tarefa_04.ipynb

Questão 1

Bloco 1: Importações e Definição da Sequência

Neste bloco, importamos as bibliotecas necessárias e definimos o termo geral da sequência $a(n)$ e o índice inicial n_0 .

```
In [8]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Definição do índice inicial  $n_0$  e da função do termo geral  $a(n)$ 
 $n_0 = 1$ 

def a(n):
    # Exemplo configurado: Série geométrica  $1/(2^n)$ 
    return 1 / (2**n)
```

Bloco 2: Coleta das Entradas do Usuário

Aqui o programa implementa a lógica para as opções (a) e (b), coletando os parâmetros de convergência, tolerância e o intervalo de k .

```
In [7]: # Escolha da opção exclusiva de execução
print("O usuário sabe que a série é convergente para uma soma S?")
print("(a) Sim")
print("(b) Não")
opcao = input("Escolha a opção (a ou b): ").strip().lower()

# Coleta de variáveis baseadas na escolha
if opcao == 'a':
    S = float(input("Digite o valor da soma S (ex: 1.0): "))
    epsilon = float(input("Digite o valor da tolerância epsilon (ex: 0.05)"))
else:
    S, epsilon = None, None

# Coleta dos limites kmin e kmax
kmin = int(input(f"Digite o valor de kmin (>= 0): "))
kmax = int(input(f"Digite o valor de kmax (> {kmin}), sugestão: diferença"))

O usuário sabe que a série é convergente para uma soma S?
(a) Sim
(b) Não
```

Bloco 3: Processamento e Geração das Tabelas (Itens i e ii)

A soma parcial $S(k)$ é calculada desde $k=0$ para garantir a acumulação correta, mas as tabelas armazenarão apenas os valores no intervalo de $[k_{\min}, k_{\max}]$.

```
In [8]: k_vals = []
a_vals = []
S_vals = []

soma_atual = 0

# Iteramos de 0 até kmax para acumular a soma corretamente desde n0
for k in range(0, kmax + 1):
    n_atual = n0 + k
    termo = a(n_atual)
    soma_atual += termo

    # Armazenamos apenas os pares dentro do intervalo solicitado
    if k >= kmin:
        k_vals.append(k)
        a_vals.append(termo)
        S_vals.append(soma_atual)

# Estruturação dos dados usando Pandas
df = pd.DataFrame({
    'k': k_vals,
    'a(k)': a_vals,
    'S(k)': S_vals
})

# Saídas (i) e (ii): Impressão das tabelas
print("\n--- (i) Tabela de pares (k, a(k)) ---")
print(df[['k', 'a(k)']].to_string(index=False))

print("\n--- (ii) Tabela de pares (k, S(k)) ---")
print(df[['k', 'S(k)']].to_string(index=False))
```

--- (i) Tabela de pares (k, a(k)) ---

k	a(k)
0	5.000000e-01
1	2.500000e-01
2	1.250000e-01
3	6.250000e-02
4	3.125000e-02
5	1.562500e-02
6	7.812500e-03
7	3.906250e-03
8	1.953125e-03
9	9.765625e-04
10	4.882812e-04
11	2.441406e-04
12	1.220703e-04
13	6.103516e-05
14	3.051758e-05
15	1.525879e-05
16	7.629395e-06
17	3.814697e-06
18	1.907349e-06
19	9.536743e-07
20	4.768372e-07
21	2.384186e-07
22	1.192093e-07
23	5.960464e-08
24	2.980232e-08
25	1.490116e-08
26	7.450581e-09
27	3.725290e-09
28	1.862645e-09
29	9.313226e-10
30	4.656613e-10
31	2.328306e-10
32	1.164153e-10
33	5.820766e-11
34	2.910383e-11
35	1.455192e-11
36	7.275958e-12
37	3.637979e-12
38	1.818989e-12
39	9.094947e-13
40	4.547474e-13
41	2.273737e-13
42	1.136868e-13
43	5.684342e-14
44	2.842171e-14
45	1.421085e-14
46	7.105427e-15
47	3.552714e-15
48	1.776357e-15
49	8.881784e-16
50	4.440892e-16
51	2.220446e-16
52	1.110223e-16
53	5.551115e-17
54	2.775558e-17
55	1.387779e-17
56	6.938894e-18
57	3.469447e-18
58	1.734723e-18
59	8.673617e-19
60	4.336809e-19

```
61 2.168404e-19
62 1.084202e-19
63 5.421011e-20
64 2.710505e-20
65 1.355253e-20
66 6.776264e-21
67 3.388132e-21
68 1.694066e-21
69 8.470329e-22
70 4.235165e-22
71 2.117582e-22
72 1.058791e-22
73 5.293956e-23
74 2.646978e-23
75 1.323489e-23
76 6.617445e-24
77 3.308722e-24
78 1.654361e-24
79 8.271806e-25
80 4.135903e-25
81 2.067952e-25
82 1.033976e-25
83 5.169879e-26
84 2.584939e-26
85 1.292470e-26
86 6.462349e-27
87 3.231174e-27
88 1.615587e-27
89 8.077936e-28
90 4.038968e-28
91 2.019484e-28
92 1.009742e-28
93 5.048710e-29
94 2.524355e-29
95 1.262177e-29
96 6.310887e-30
97 3.155444e-30
98 1.577722e-30
99 7.888609e-31
100 3.944305e-31
```

--- (ii) Tabela de pares (k, S(k)) ---

k	S(k)
0	0.500000
1	0.750000
2	0.875000
3	0.937500
4	0.968750
5	0.984375
6	0.992188
7	0.996094
8	0.998047
9	0.999023
10	0.999512
11	0.999756
12	0.999878
13	0.999939
14	0.999969
15	0.999985
16	0.999992
17	0.999996
18	0.999998
19	0.999999

20 1.000000
21 1.000000
22 1.000000
23 1.000000
24 1.000000
25 1.000000
26 1.000000
27 1.000000
28 1.000000
29 1.000000
30 1.000000
31 1.000000
32 1.000000
33 1.000000
34 1.000000
35 1.000000
36 1.000000
37 1.000000
38 1.000000
39 1.000000
40 1.000000
41 1.000000
42 1.000000
43 1.000000
44 1.000000
45 1.000000
46 1.000000
47 1.000000
48 1.000000
49 1.000000
50 1.000000
51 1.000000
52 1.000000
53 1.000000
54 1.000000
55 1.000000
56 1.000000
57 1.000000
58 1.000000
59 1.000000
60 1.000000
61 1.000000
62 1.000000
63 1.000000
64 1.000000
65 1.000000
66 1.000000
67 1.000000
68 1.000000
69 1.000000
70 1.000000
71 1.000000
72 1.000000
73 1.000000
74 1.000000
75 1.000000
76 1.000000
77 1.000000
78 1.000000
79 1.000000
80 1.000000
81 1.000000
82 1.000000

```

83 1.000000
84 1.000000
85 1.000000
86 1.000000
87 1.000000
88 1.000000
89 1.000000
90 1.000000
91 1.000000
92 1.000000
93 1.000000
94 1.000000
95 1.000000
96 1.000000
97 1.000000
98 1.000000
99 1.000000
100 1.000000

```

Bloco 4: Plotagem do Gráfico (Itens iii e iv)

Este bloco gera a visualização discreta dos pontos. Se a opção (a) tiver sido escolhida, ele adiciona as linhas horizontais de limite e verifica automaticamente se a condição de ter "pelo menos a metade dos pares na faixa" foi atingida.

```

In [9]: plt.figure(figsize=(10, 6))

# (iii) Gráfico mostrando os conjuntos de pontos em cores distintas de ma
plt.scatter(df['k'], df['a(k)'], color='blue', label='Termo: a(k)', zorde
plt.scatter(df['k'], df['S(k)'], color='red', label='Soma Parcial: S(k)',

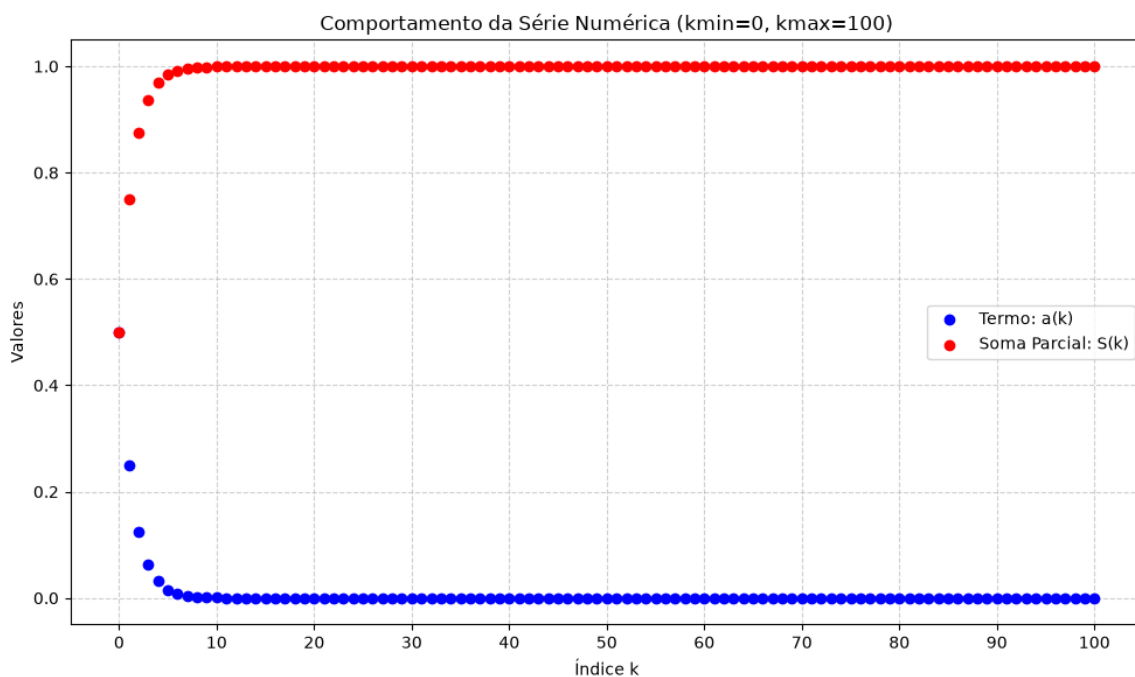
# (iv) Adição das retas horizontais caso a opção seja (a)
if opcao == 'a':
    plt.hlines(y=S, xmin=kmin, xmax=kmax, colors='green', linestyle='sol
    plt.hlines(y=S - epsilon, xmin=kmin, xmax=kmax, colors='orange', line
    plt.hlines(y=S + epsilon, xmin=kmin, xmax=kmax, colors='orange', line

    # Verificação auxiliar: checar se metade dos pontos S(k) estão na fai
    pontos_na_faixa = sum((S - epsilon) <= val <= (S + epsilon) for val i
    if pontos_na_faixa < len(S_vals) / 2:
        print(f"\nAVISO: Apenas {pontos_na_faixa} de {len(S_vals)} pontos
        print("Ajuste os valores de kmin, kmax ou epsilon na próxima exec

# Configurações visuais do gráfico
plt.title(f"Comportamento da Série Numérica (kmin={kmin}, kmax={kmax})")
plt.xlabel("Índice k")
plt.ylabel("Valores")
plt.xticks(np.arange(kmin, kmax + 1, step=max(1, (kmax - kmin) // 10)))
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()

# Exibição da figura
plt.show()

```



Questão 2(a)

In []: `n0 = 1`

```
def a(n):
    return 1 / n
```

```
In [12]: # Coleta das entradas
print("O usuário sabe que a série é convergente para uma soma S?")
print("(a) Sim")
print("(b) Não")
opcao = input("Escolha a opção (a ou b): ").strip().lower()

if opcao == 'a':
    S = float(input("Digite o valor da soma S (ex: 1.0): "))
    epsilon = float(input("Digite o valor da tolerância epsilon (ex: 0.05)"))
else:
    S, epsilon = None, None

kmin = int(input(f"Digite o valor de kmin (>= 0): "))
kmax = int(input(f"Digite o valor de kmax (> {kmin}): "))
```

O usuário sabe que a série é convergente para uma soma S?

(a) Sim

(b) Não

```
In [13]: k_vals = []
a_vals = []
S_vals = []
soma_atual = 0

# Processamento
for k in range(0, kmax + 1):
    n_atual = n0 + k
    termo = a(n_atual)
    soma_atual += termo

    if k >= kmin:
        k_vals.append(k)
        a_vals.append(termo)
        S_vals.append(soma_atual)

# Criação do DataFrame
df = pd.DataFrame({'k': k_vals, 'a(k)': a_vals, 'S(k)': S_vals})
```

```
In [14]: # Saídas (i) e (ii) renderizadas para Jupyter Notebook
print("--- (i) Tabela de pares (k, a(k)) ---")
display(df[['k', 'a(k)']])

print("\n--- (ii) Tabela de pares (k, S(k)) ---")
display(df[['k', 'S(k)']])
```

--- (i) Tabela de pares (k, a(k)) ---

	k	a(k)
0	0	1.000000
1	1	0.500000
2	2	0.333333
3	3	0.250000
4	4	0.200000
5	5	0.166667
6	6	0.142857
7	7	0.125000
8	8	0.111111
9	9	0.100000
10	10	0.090909

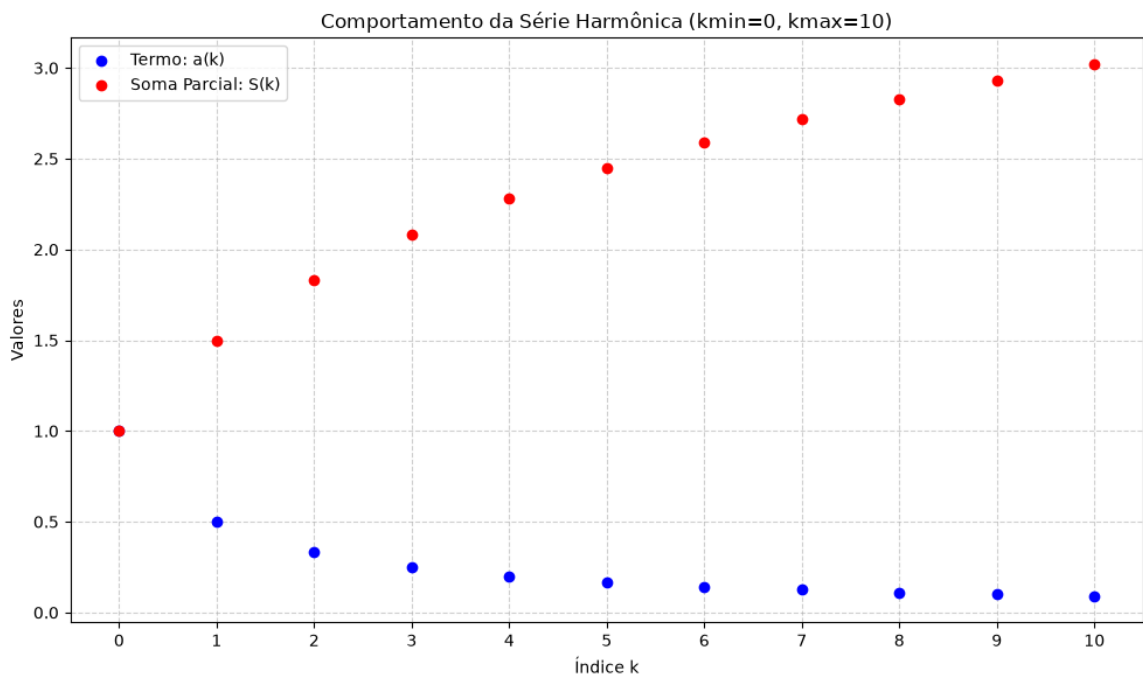
--- (ii) Tabela de pares (k, S(k)) ---

	k	S(k)
0	0	1.000000
1	1	1.500000
2	2	1.833333
3	3	2.083333
4	4	2.283333
5	5	2.450000
6	6	2.592857
7	7	2.717857
8	8	2.828968
9	9	2.928968
10	10	3.019877

```
In [15]: # Saídas (iii) e (iv) na interface gráfica
plt.figure(figsize=(10, 6))
plt.scatter(df['k'], df['a(k)'], color='blue', label='Termo: a(k)', zorder=1)
plt.scatter(df['k'], df['S(k)'], color='red', label='Soma Parcial: S(k)', zorder=2)

if opcao == 'a':
    plt.hlines(y=S, xmin=kmin, xmax=kmax, colors='green', linestyle='solid')
    plt.hlines(y=S - epsilon, xmin=kmin, xmax=kmax, colors='orange', linestyle='dashed')
    plt.hlines(y=S + epsilon, xmin=kmin, xmax=kmax, colors='orange', linestyle='dashed')

plt.title(f"Comportamento da Série Harmônica (kmin={kmin}, kmax={kmax})")
plt.xlabel("Índice k")
plt.ylabel("Valores")
plt.xticks(np.arange(kmin, kmax + 1, step=max(1, (kmax - kmin) // 10)))
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



Questão 4(c)

In [9]: `import numpy as np`

```
# n0 = 1
def a(n):
    # Expressão original da série
    return 1 / (np.sqrt(n+1) * np.sqrt(n) * (np.sqrt(n+1) + np.sqrt(n)))
```

In [3]: `# Coleta das entradas`

```
print("O usuário sabe que a série é convergente para uma soma S?")
print("(a) Sim")
print("(b) Não")
opcao = input("Escolha a opção (a ou b): ").strip().lower()

if opcao == 'a':
    S = float(input("Digite o valor da soma S (ex: 1.0): "))
    epsilon = float(input("Digite o valor da tolerância epsilon (ex: 0.05)"))
else:
    S, epsilon = None, None

kmin = int(input(f"Digite o valor de kmin (>= 0): "))
kmax = int(input(f"Digite o valor de kmax (> {kmin}): "))
```

O usuário sabe que a série é convergente para uma soma S?

(a) Sim

(b) Não

```
In [10]: k_vals = []
a_vals = []
S_vals = []
soma_atual = 0
n0 = 1

# Processamento
for k in range(0, kmax + 1):
    n_atual = n0 + k
    termo = a(n_atual)
    soma_atual += termo

    if k >= kmin:
        k_vals.append(k)
        a_vals.append(termo)
        S_vals.append(soma_atual)

# Criação do DataFrame
df = pd.DataFrame({'k': k_vals, 'a(k)': a_vals, 'S(k)': S_vals})
```

```
In [11]: # Saídas (i) e (ii) renderizadas para Jupyter Notebook
print("--- (i) Tabela de pares (k, a(k)) ---")
display(df[['k', 'a(k)']])

print("\n--- (ii) Tabela de pares (k, S(k)) ---")
display(df[['k', 'S(k)']])

--- (i) Tabela de pares (k, a(k)) ---
```

	k	a(k)
0	999990	5.000064e-10
1	999991	5.000056e-10
2	999992	5.000049e-10
3	999993	5.000041e-10
4	999994	5.000034e-10
5	999995	5.000026e-10
6	999996	5.000019e-10
7	999997	5.000011e-10
8	999998	5.000004e-10
9	999999	4.999996e-10
10	1000000	4.999989e-10
11	1000001	4.999981e-10
12	1000002	4.999974e-10
13	1000003	4.999966e-10
14	1000004	4.999959e-10
15	1000005	4.999951e-10
16	1000006	4.999944e-10
17	1000007	4.999936e-10
18	1000008	4.999929e-10
19	1000009	4.999921e-10
20	1000010	4.999914e-10

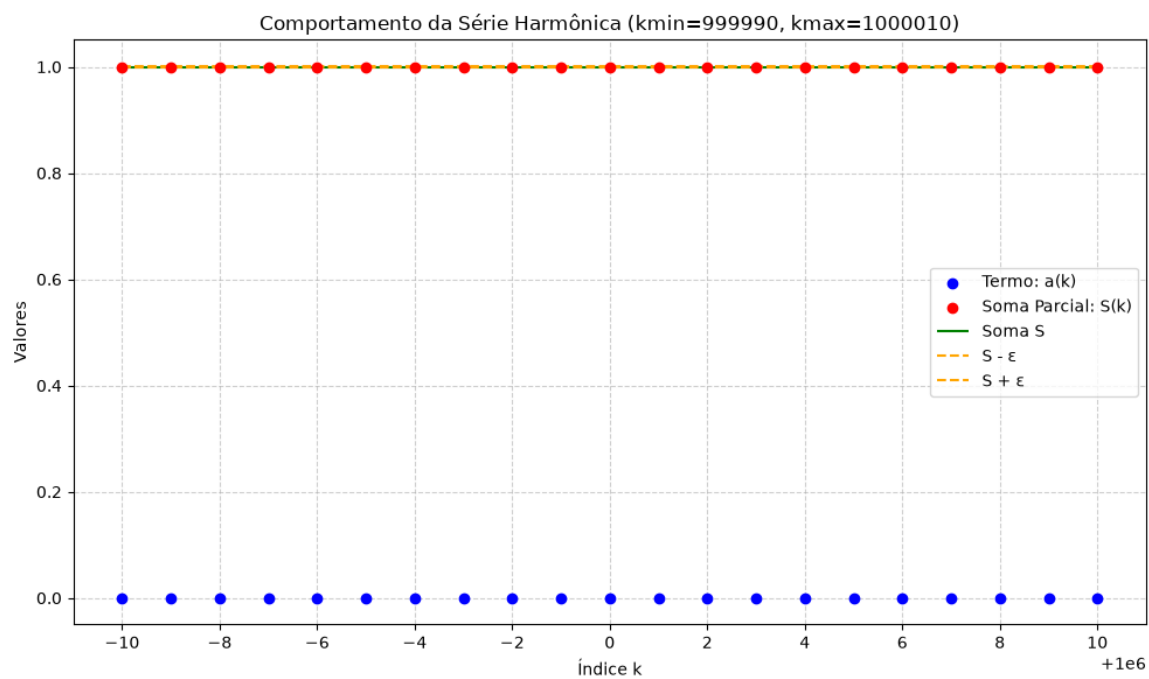
--- (ii) Tabela de pares (k, S(k)) ---

	k	S(k)
0	999990	0.999
1	999991	0.999
2	999992	0.999
3	999993	0.999
4	999994	0.999
5	999995	0.999
6	999996	0.999
7	999997	0.999
8	999998	0.999
9	999999	0.999
10	1000000	0.999
11	1000001	0.999
12	1000002	0.999
13	1000003	0.999
14	1000004	0.999
15	1000005	0.999
16	1000006	0.999
17	1000007	0.999
18	1000008	0.999
19	1000009	0.999
20	1000010	0.999

```
In [12]: # Saídas (iii) e (iv) na interface gráfica
plt.figure(figsize=(10, 6))
plt.scatter(df['k'], df['a(k)'], color='blue', label='Termo: a(k)', zorder=1)
plt.scatter(df['k'], df['S(k)'], color='red', label='Soma Parcial: S(k)', zorder=2)

if opcao == 'a':
    plt.hlines(y=S, xmin=kmin, xmax=kmax, colors='green', linestyle='solid')
    plt.hlines(y=S - epsilon, xmin=kmin, xmax=kmax, colors='orange', linestyle='dashed')
    plt.hlines(y=S + epsilon, xmin=kmin, xmax=kmax, colors='orange', linestyle='dashed')

plt.title(f"Comportamento da Série Harmônica (kmin={kmin}, kmax={kmax})")
plt.xlabel("Índice k")
plt.ylabel("Valores")
plt.xticks(np.arange(kmin, kmax + 1, step=max(1, (kmax - kmin) // 10)))
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



In []:

Questão 3. a) Se $\lim a_n = 0$, então $\sum_{n=n_0}^{\infty} a_n$ converge

• Falso.

Exemplo: contraexemplo: Usar o contraexemplo mais clássico para esta afirmação é a série Harmônica $a_n = \frac{1}{n}$

$\lim_{n \rightarrow \infty} \frac{1}{n} = 0$; Porém $\sum_{n=1}^{\infty} \frac{1}{n}$ é divergente, pois sua soma tende ao

infinito, pois $\lim a_n = 0$, logo, os termos ficarem cada vez menores, não garante que a soma de infinitos resulte em um finito

Conclusão = Se $\sum_{n=n_0}^{\infty} a_n$ converge, então $\lim a_n = 0$

Questão 4. c

• Termo Enal $a_n = \frac{1}{\sqrt{n+1} \cdot \sqrt{n} \cdot (\sqrt{n+1} + \sqrt{n})}$

Exemplo: Identificar a série e racionalizar o denominador

$$a_n = \frac{1 \cdot (\sqrt{n+1} - \sqrt{n})}{\sqrt{n+1} \cdot \sqrt{n} \cdot (\sqrt{n+1} + \sqrt{n}) \cdot (\sqrt{n+1} - \sqrt{n})} = \frac{\sqrt{n+1} - \sqrt{n}}{\sqrt{n+1} \cdot \sqrt{n} \cdot 1}$$

$$a_n = \frac{\cancel{\sqrt{n+1}}}{\cancel{\sqrt{n+1}} \cdot \sqrt{n}} - \frac{\cancel{\sqrt{n}}}{\sqrt{n+1} \cdot \cancel{\sqrt{n}}} = \frac{1}{\sqrt{n}} - \frac{1}{\sqrt{n+1}}$$

Exercício 2: Cálculo Analítico da Soma

$$S_k = \sum_{n=1}^k \left(\frac{1}{\sqrt{n}} - \frac{1}{\sqrt{n+1}} \right)$$

$$S_k = \left(\frac{1}{\sqrt{1}} - \frac{1}{\sqrt{2}} \right) + \left(\frac{1}{\sqrt{2}} - \frac{1}{\sqrt{3}} \right) + \left(\frac{1}{\sqrt{3}} - \frac{1}{\sqrt{4}} \right) + \dots + \left(\frac{1}{\sqrt{k}} - \frac{1}{\sqrt{k+1}} \right)$$

$$S_k = 1 - \frac{1}{\sqrt{k+1}}$$

• calculando o limite para encontrar a soma finita de S

$$S = \lim_{k \rightarrow \infty} \left(1 - \frac{1}{\sqrt{k+1}} \right) = S = 1 - 0 = 1$$

$\sqrt{k+1} \rightarrow 0$

Exercício 3: Parâmetros Para o Programa

$$|S_k - S| \leq 0.001$$

$$\left| \left(1 - \frac{1}{\sqrt{k+1}} \right) - 1 \right| \leq 0.001$$

$$\frac{1}{\sqrt{k+1}} \leq \frac{1}{1000}$$

$$\sqrt{k+1} \geq 1000 \Rightarrow k+1 \geq 1\,000\,000 \Rightarrow k \geq 999\,999$$

Para Opção (A)

$$S = 1.0$$

$$\varepsilon = 0.001$$

$$K_{\min} = 999990$$

$$K_{\max} = 1000010$$

--- (i) Tabela de pares (k, a(k)) ---

	k	a(k)
0	999990	5.000064e-10
1	999991	5.000056e-10
2	999992	5.000049e-10
3	999993	5.000041e-10
4	999994	5.000034e-10
5	999995	5.000026e-10
6	999996	5.000019e-10
7	999997	5.000011e-10
8	999998	5.000004e-10
9	999999	4.999996e-10
10	1000000	4.999989e-10
11	1000001	4.999981e-10
12	1000002	4.999974e-10
13	1000003	4.999966e-10
14	1000004	4.999959e-10
15	1000005	4.999951e-10
16	1000006	4.999944e-10
17	1000007	4.999936e-10
18	1000008	4.999929e-10
19	1000009	4.999921e-10

--- (ii) Tabela de pares (k, S(k)) ---

	k	S(k)
0	999990	0.999
1	999991	0.999
2	999992	0.999
3	999993	0.999
4	999994	0.999
5	999995	0.999
6	999996	0.999
7	999997	0.999
8	999998	0.999
9	999999	0.999
10	1000000	0.999
11	1000001	0.999
12	1000002	0.999
13	1000003	0.999
14	1000004	0.999
15	1000005	0.999
16	1000006	0.999
17	1000007	0.999
18	1000008	0.999
19	1000009	0.999
20	1000010	0.999

